

| Aspect                 | Collections Framework                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
|------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Java Version           | Java 1.2                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
| Year                   | 1998                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
| Feature                | Collections Framework                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
| UniqueClassName        | CollectionsFramework                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
| Definition             | The Java Collections Framework is a unified architecture for storing, accessing, and manipulating groups of objects. It provides interfaces such as Collection, List, Set, Queue, and Map, along with implementations such as ArrayList, LinkedList, HashSet, TreeSet, HashMap, and TreeMap.                                                                                                                                                                                                                                                                                                                                        |
| Main Purpose           | To provide reusable, standardized data structures and algorithms instead of requiring developers to implement common collection structures themselves.                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
| Key Interfaces         | List, Set, Queue, Deque, Map, SortedSet, SortedMap                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
| Common Implementations | ArrayList, LinkedList, HashSet, LinkedHashSet, TreeSet, PriorityQueue, ArrayDeque, HashMap, LinkedHashMap, TreeMap                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
| Advantages             | <ol style="list-style-type: none"> <li>1. Standardized data structures</li> <li>2. Reduces development effort</li> <li>3. Provides reusable algorithms through Collections utility methods</li> <li>4. Supports dynamic sizing</li> <li>5. Provides multiple implementations for different performance requirements</li> <li>6. Supports sorting, searching, reversing, shuffling, etc.</li> <li>7. Improves code maintainability</li> <li>8. Generics provide compile-time type safety</li> <li>9. Well-tested and widely used API</li> </ol>                                                                                      |
| Disadvantages          | <ol style="list-style-type: none"> <li>1. Collections generally store objects rather than primitive values, so boxing/unboxing may be involved</li> <li>2. Some implementations have memory overhead compared with primitive arrays</li> <li>3. Choosing the wrong implementation can hurt performance</li> <li>4. Thread safety is not automatically provided by most modern collection implementations</li> <li>5. Complex collection types can make code harder to understand</li> <li>6. Some collections have synchronization/concurrency considerations that require specialized classes such as ConcurrentHashMap</li> </ol> |

|                             |                                                                                                                                                                                                                   |
|-----------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>When to Use</b>          | Use when you need to store and manipulate a variable number of objects, especially when you need operations such as searching, sorting, uniqueness, insertion/removal, key-value lookup, queues, or ordered data. |
| <b>When NOT to Use</b>      | Avoid when a simple fixed-size primitive array is sufficient, when extremely low memory overhead is critical, or when specialized primitive collections are required for very large numerical datasets.           |
| <b>Real-Time Use Case 1</b> | <b>Shopping cart:</b> <code>ArrayList&lt;Product&gt;</code> can maintain products added to a customer's cart.                                                                                                     |
| <b>Example 1</b>            | <code>List&lt;Product&gt; cart = new ArrayList&lt;&gt;();</code>                                                                                                                                                  |
| <b>Real-Time Use Case 2</b> | <b>Unique user roles:</b> <code>HashSet&lt;String&gt;</code> can maintain unique roles such as ADMIN, USER, and MANAGER.                                                                                          |
| <b>Example 2</b>            | <code>Set&lt;String&gt; roles = new HashSet&lt;&gt;();</code>                                                                                                                                                     |
| <b>Real-Time Use Case 3</b> | <b>Product lookup:</b> <code>HashMap&lt;Long, Product&gt;</code> can associate a product ID with its product object.                                                                                              |
| <b>Example 3</b>            | <code>Map&lt;Long, Product&gt; products = new HashMap&lt;&gt;();</code>                                                                                                                                           |
| <b>Real-Time Use Case 4</b> | <b>Task processing:</b> <code>Queue&lt;Task&gt;</code> or <code>Deque&lt;Task&gt;</code> can manage tasks waiting to be processed.                                                                                |
| <b>Example 4</b>            | <code>Queue&lt;Task&gt; tasks = new ArrayDeque&lt;&gt;();</code>                                                                                                                                                  |
| <b>Real-Time Use Case 5</b> | <b>Priority processing:</b> <code>PriorityQueue&lt;Task&gt;</code> can process high-priority jobs before lower-priority jobs.                                                                                     |
| <b>Example 5</b>            | <code>Queue&lt;Task&gt; tasks = new PriorityQueue&lt;&gt;();</code>                                                                                                                                               |
| <b>Real-Time Use Case 6</b> | <b>Leaderboards/rankings:</b> <code>TreeSet</code> or <code>TreeMap</code> can maintain elements in sorted order.                                                                                                 |
| <b>Example 6</b>            | <code>TreeSet&lt;Integer&gt; scores = new TreeSet&lt;&gt;();</code>                                                                                                                                               |
| <b>Real-Time Use Case 7</b> | <b>Caching/session lookup:</b> <code>HashMap</code> or <code>ConcurrentHashMap</code> can provide fast key-based lookup.                                                                                          |
| <b>Example 7</b>            | <code>Map&lt;String, UserSession&gt; sessions = new ConcurrentHashMap&lt;&gt;();</code>                                                                                                                           |
| <b>Alternate Feature</b>    | <b>Arrays</b> are the simplest alternative when the number of elements is fixed and indexed access is the primary requirement. For specialized high-performance primitive data, specialized primitive             |

|                                       |                                                                                                                                                                                                                              |
|---------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                       | collections from third-party libraries can also be considered.                                                                                                                                                               |
| <b>Collections Framework vs Array</b> | Collections are preferable for dynamic size, specialized operations, and reusable data structures; arrays are preferable for fixed-size data, simple indexed access, and lower overhead.                                     |
| <b>Best Practice</b>                  | Program to the interface: prefer List, Set, Queue, or Map as the variable type and select the appropriate implementation based on required ordering, uniqueness, lookup, insertion/removal, and concurrency characteristics. |

### Quick implementation-selection table

| Requirement                                    | Recommended Collection | Example                     |
|------------------------------------------------|------------------------|-----------------------------|
| <b>Dynamic ordered list</b>                    | ArrayList              | Shopping cart               |
| <b>Frequent insertion/removal at ends</b>      | ArrayDeque             | Task/deque processing       |
| <b>Need unique elements</b>                    | HashSet                | Unique user IDs             |
| <b>Unique + insertion order</b>                | LinkedHashSet          | Recently added unique items |
| <b>Unique + sorted</b>                         | TreeSet                | Sorted scores               |
| <b>Key-value lookup</b>                        | HashMap                | Product ID → Product        |
| <b>Key-value + insertion order</b>             | LinkedHashMap          | Ordered cache data          |
| <b>Key-value + sorted keys</b>                 | TreeMap                | Sorted employee IDs         |
| <b>Priority-based processing</b>               | PriorityQueue          | Job scheduler               |
| <b>Thread-safe concurrent key-value access</b> | ConcurrentHashMap      | Session/cache store         |
| <b>Fixed-size, primitive-focused data</b>      | Array                  | int[], double[]             |

**Important:** The Collections Framework was introduced in **Java 1.2 (1998)** and largely replaced the older collection-related classes and interfaces such as Vector, Stack, and Hashtable as the preferred general-purpose collection architecture.

| Aspect               | List / ArrayList                                                                                                                                                                                                                                                                                                                                                                  |
|----------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Java Version         | Java 1.2                                                                                                                                                                                                                                                                                                                                                                          |
| Year                 | 1998                                                                                                                                                                                                                                                                                                                                                                              |
| Feature              | List / ArrayList                                                                                                                                                                                                                                                                                                                                                                  |
| UniqueClassName      | ArrayList                                                                                                                                                                                                                                                                                                                                                                         |
| Definition           | List is an interface in the Java Collections Framework that represents an <b>ordered collection of elements</b> . ArrayList is a resizable-array implementation of List. It allows duplicate elements and maintains insertion order.                                                                                                                                              |
| Basic Syntax         | List<Type> list = new ArrayList<>();                                                                                                                                                                                                                                                                                                                                              |
| Underlying Structure | ArrayList uses a <b>dynamically resizable array</b> internally.                                                                                                                                                                                                                                                                                                                   |
| Duplicates           | Allowed                                                                                                                                                                                                                                                                                                                                                                           |
| Insertion Order      | Maintained                                                                                                                                                                                                                                                                                                                                                                        |
| Null Values          | Allowed                                                                                                                                                                                                                                                                                                                                                                           |
| Index-Based Access   | Supported; typically very fast, <b>O(1)</b> for get() and set().                                                                                                                                                                                                                                                                                                                  |
| Size                 | Dynamic; automatically grows as elements are added.                                                                                                                                                                                                                                                                                                                               |
| Thread Safety        | Not synchronized by default. For concurrent modification requirements, consider alternatives such as CopyOnWriteArrayList or appropriate external synchronization.                                                                                                                                                                                                                |
| Advantages           | <ol style="list-style-type: none"><li>1. Fast index-based access.</li><li>2. Dynamic/resizable size.</li><li>3. Maintains insertion order.</li><li>4. Allows duplicate elements.</li><li>5. Allows null values.</li><li>6. Provides convenient methods such as add(), remove(), contains(), get(), and set().</li><li>7. Works naturally with generics for type safety.</li></ol> |

|                             |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
|-----------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                             | 8. Excellent general-purpose choice when reads are more frequent than middle-of-list insertions/removals.                                                                                                                                                                                                                                                                                                                                                                                                      |
| <b>Disadvantages</b>        | 1. Inserting/removing elements in the middle can be expensive because elements may need to be shifted ( <b><math>O(n)</math></b> ).<br>2. Resizing can require allocation of a larger internal array and copying elements.<br>3. Uses more memory than a primitive array in some scenarios because it stores object references and may involve boxing for primitives.<br>4. Not thread-safe by default.<br>5. Poor choice when frequent insertion/removal occurs near the beginning or middle of a large list. |
| <b>When to Use</b>          | Use ArrayList when you need an <b>ordered, dynamically sized collection</b> and frequently access elements by index. It is especially suitable when additions are mostly at the end and reads/searches are common.                                                                                                                                                                                                                                                                                             |
| <b>When NOT to Use</b>      | Avoid ArrayList when frequent insertions/deletions occur near the beginning or middle of a large list, when strict concurrent modification requirements need a specialized thread-safe collection, or when a simple fixed-size primitive array is more appropriate.                                                                                                                                                                                                                                            |
| <b>Real-Time Use Case 1</b> | <b>Shopping cart:</b> Store products in the order they were added.                                                                                                                                                                                                                                                                                                                                                                                                                                             |
| <b>Example 1</b>            | List<Product> cart = new ArrayList<>();                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
| <b>Real-Time Use Case 2</b> | <b>Student records:</b> Maintain a dynamically growing list of students.                                                                                                                                                                                                                                                                                                                                                                                                                                       |
| <b>Example 2</b>            | List<Student> students = new ArrayList<>();                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
| <b>Real-Time Use Case 3</b> | <b>Order items:</b> Store items belonging to an e-commerce order.                                                                                                                                                                                                                                                                                                                                                                                                                                              |
| <b>Example 3</b>            | List<OrderItem> items = new ArrayList<>();                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
| <b>Real-Time Use Case 4</b> | <b>Playlist:</b> Maintain songs in playback order and access a song by position.                                                                                                                                                                                                                                                                                                                                                                                                                               |
| <b>Example 4</b>            | List<Song> playlist = new ArrayList<>();                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
| <b>Real-Time Use Case 5</b> | <b>Search results:</b> Store a variable number of returned records while preserving their order.                                                                                                                                                                                                                                                                                                                                                                                                               |
| <b>Example 5</b>            | List<Result> results = new ArrayList<>();                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
| <b>Real-Time Use Case 6</b> | <b>API response data:</b> Store a collection of objects returned from an API.                                                                                                                                                                                                                                                                                                                                                                                                                                  |

|                                                           |                                                                                                                                                                                                                                                                 |
|-----------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                                           |                                                                                                                                                                                                                                                                 |
| <b>Example 6</b>                                          | List<User> users = new ArrayList<>();                                                                                                                                                                                                                           |
| <b>Real-Time Use Case 7</b>                               | <b>Recently displayed products:</b> Maintain an ordered collection of products shown to a customer.                                                                                                                                                             |
| <b>Example 7</b>                                          | List<Product> products = new ArrayList<>();                                                                                                                                                                                                                     |
| <b>Simple Example</b>                                     | List<String> names = new ArrayList<>();<br>names.add("Anurag");<br>names.add("Rahul");<br>names.add("Anurag");<br>System.out.println(names.get(1));                                                                                                             |
| <b>Output</b>                                             | Rahul                                                                                                                                                                                                                                                           |
| <b>Alternate Feature</b>                                  | <b>LinkedList</b> is the primary standard-library alternative when frequent insertions/removals are required and positional/indexed access is less important. Other alternatives include Vector, CopyOnWriteArrayList, or arrays depending on the requirements. |
| <b>Best Alternative for Fixed Size</b>                    | <b>Array</b> — e.g., String[] names = new String[10];                                                                                                                                                                                                           |
| <b>Best Alternative for Frequent Middle Insert/Delete</b> | <b>LinkedList</b> — useful when operations are performed through known list nodes/iterators, although it is generally slower than ArrayList for random access.                                                                                                  |
| <b>Best Alternative for Thread-Safe Read-Heavy Lists</b>  | <b>CopyOnWriteArrayList</b> — useful when reads greatly outnumber writes and concurrent access is required.                                                                                                                                                     |
| <b>Best Alternative for Legacy Synchronized List</b>      | <b>Vector</b> — synchronized, but generally not preferred for new code when modern concurrency alternatives are more appropriate.                                                                                                                               |

## List vs ArrayList

| Feature                          | List                                   | ArrayList                                 |
|----------------------------------|----------------------------------------|-------------------------------------------|
| <b>Type</b>                      | <b>Interface</b>                       | <b>Class</b>                              |
| <b>Purpose</b>                   | Defines the contract for ordered lists | Provides a resizable-array implementation |
| <b>Can instantiate directly?</b> | ✗ No                                   | ✓ Yes                                     |

|                                   |                             |                              |
|-----------------------------------|-----------------------------|------------------------------|
|                                   |                             |                              |
| <b>Example</b>                    | List<String> x              | new ArrayList<>()            |
| <b>Implementation flexibility</b> | High                        | Specific implementation      |
| <b>Recommended declaration</b>    | List<String> list           | ArrayList<String> list       |
| <b>Typical usage</b>              | <b>Program to interface</b> | <b>Choose implementation</b> |

### ArrayList vs Common Alternatives

| Requirement                               | Best Choice                                                       | Reason                                                       |
|-------------------------------------------|-------------------------------------------------------------------|--------------------------------------------------------------|
| <b>Fast random/index access</b>           | <b>ArrayList</b>                                                  | Efficient array-based access                                 |
| <b>Dynamic ordered collection</b>         | <b>ArrayList</b>                                                  | Resizable and maintains order                                |
| <b>Frequent additions at end</b>          | <b>ArrayList</b>                                                  | Efficient append in typical cases                            |
| <b>Frequent middle insertion/deletion</b> | <b>LinkedList</b> / another structure depending on access pattern | Avoids repeated array shifting in appropriate usage patterns |
| <b>Fixed-size primitive data</b>          | <b>Array</b>                                                      | Lower overhead and direct primitive storage                  |
| <b>Thread-safe read-heavy list</b>        | <b>CopyOnWriteArrayList</b>                                       | Designed for concurrent read-heavy workloads                 |
| <b>Unique elements</b>                    | <b>HashSet</b>                                                    | Prevents duplicates                                          |
| <b>Sorted unique elements</b>             | <b>TreeSet</b>                                                    | Maintains sorted set semantics                               |
| <b>Key-value data</b>                     | <b>HashMap</b>                                                    | Designed for key-value lookup                                |

**Key point:** List is the **interface**, while ArrayList is one of its most commonly used **implementations**. In most application code, prefer:

```
List<String> names = new ArrayList<>();
```

rather than exposing the implementation type unnecessarily.

| Aspect               | Set / HashSet                                                                                                                                                                                                                                                                                                                                                                                            |
|----------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Java Version         | Java 1.2                                                                                                                                                                                                                                                                                                                                                                                                 |
| Year                 | 1998                                                                                                                                                                                                                                                                                                                                                                                                     |
| Feature              | Set / HashSet                                                                                                                                                                                                                                                                                                                                                                                            |
| UniqueClassName      | HashSet                                                                                                                                                                                                                                                                                                                                                                                                  |
| Definition           | Set is an interface in the Java Collections Framework that represents a collection that <b>does not permit duplicate elements</b> . HashSet is a hash-table-based implementation of Set.                                                                                                                                                                                                                 |
| Basic Syntax         | Set<Type> set = new HashSet<>();                                                                                                                                                                                                                                                                                                                                                                         |
| Underlying Structure | HashSet is backed by a HashMap internally.                                                                                                                                                                                                                                                                                                                                                               |
| Duplicates           | ✗ Not allowed                                                                                                                                                                                                                                                                                                                                                                                            |
| Ordering             | ✗ No guaranteed iteration order                                                                                                                                                                                                                                                                                                                                                                          |
| Null Values          | ✓ Allows one null element                                                                                                                                                                                                                                                                                                                                                                                |
| Index-Based Access   | ✗ Not supported                                                                                                                                                                                                                                                                                                                                                                                          |
| Typical Performance  | add(), remove(), and contains() are typically <b>O(1)</b> on average, assuming good hash distribution.                                                                                                                                                                                                                                                                                                   |
| Thread Safety        | Not synchronized by default. For concurrent use, consider appropriate concurrent collections or synchronization.                                                                                                                                                                                                                                                                                         |
| Advantages           | <ol style="list-style-type: none"><li>1. Automatically prevents duplicate elements.</li><li>2. Fast average add(), remove(), and contains() operations.</li><li>3. Useful for membership testing.</li><li>4. Dynamically grows as elements are added.</li><li>5. Simple API and easy to use.</li><li>6. Supports generics for compile-time type safety.</li><li>7. Can store one null element.</li></ol> |
| Disadvantages        | <ol style="list-style-type: none"><li>1. Does not guarantee insertion order.</li><li>2. Does not provide index-based access.</li></ol>                                                                                                                                                                                                                                                                   |



|                             |                                                                                                                                                                                                                                                  |
|-----------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                             | 3. Hashing introduces memory overhead compared with a simple array.<br>4. Performance can degrade with poor hash-code distribution.<br>5. Not thread-safe by default.<br>6. Not suitable when sorted or predictable iteration order is required. |
| <b>When to Use</b>          | Use HashSet when <b>uniqueness and fast membership checking</b> are more important than ordering or indexed access.                                                                                                                              |
| <b>When NOT to Use</b>      | Do not use HashSet when you need duplicate elements, reliable insertion order, sorted order, or frequent index-based access.                                                                                                                     |
| <b>Real-Time Use Case 1</b> | <b>Unique user IDs:</b> Prevent the same user ID from being registered or processed twice.                                                                                                                                                       |
| <b>Example 1</b>            | <code>Set&lt;Long&gt; userIds = new HashSet&lt;&gt;();</code>                                                                                                                                                                                    |
| <b>Real-Time Use Case 2</b> | <b>Unique product categories:</b> Store categories without duplicates.                                                                                                                                                                           |
| <b>Example 2</b>            | <code>Set&lt;String&gt; categories = new HashSet&lt;&gt;();</code>                                                                                                                                                                               |
| <b>Real-Time Use Case 3</b> | <b>Permission management:</b> Store unique permissions such as READ, WRITE, and DELETE.                                                                                                                                                          |
| <b>Example 3</b>            | <code>Set&lt;String&gt; permissions = new HashSet&lt;&gt;();</code>                                                                                                                                                                              |
| <b>Real-Time Use Case 4</b> | <b>Removing duplicate data:</b> Convert a list containing repeated values into a collection of unique values.                                                                                                                                    |
| <b>Example 4</b>            | <code>Set&lt;String&gt; uniqueNames = new HashSet&lt;&gt;(names);</code>                                                                                                                                                                         |
| <b>Real-Time Use Case 5</b> | <b>Visited pages/URLs:</b> Track pages already visited so that they are not processed again.                                                                                                                                                     |
| <b>Example 5</b>            | <code>Set&lt;String&gt; visitedUrls = new HashSet&lt;&gt;();</code>                                                                                                                                                                              |
| <b>Real-Time Use Case 6</b> | <b>Event processing:</b> Track event IDs that have already been processed to avoid duplicate processing.                                                                                                                                         |
| <b>Example 6</b>            | <code>Set&lt;String&gt; processedEvents = new HashSet&lt;&gt;();</code>                                                                                                                                                                          |
| <b>Real-Time Use Case 7</b> | <b>Unique tags:</b> Maintain unique tags associated with an article, product, or post.                                                                                                                                                           |
| <b>Example 7</b>            | <code>Set&lt;String&gt; tags = new HashSet&lt;&gt;();</code>                                                                                                                                                                                     |
| <b>Simple Example</b>       | <code>Set&lt;String&gt; names = new</code>                                                                                                                                                                                                       |

|                                       |                                                                                                                                                                 |
|---------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                       | <code>HashSet&lt;&gt;();names.add("Anurag");names.add("Rahul");names.add("Anurag");</code>                                                                      |
| <b>Result</b>                         | The set contains only Anurag and Rahul; the second "Anurag" is ignored.                                                                                         |
| <b>Alternate Feature</b>              | <b>LinkedHashSet</b> when you need uniqueness <b>plus insertion order</b> ; <b>TreeSet</b> when you need uniqueness <b>plus sorted order</b> .                  |
| <b>Alternative for Indexed Data</b>   | <b>ArrayList</b> when duplicates are acceptable and indexed access/order are important.                                                                         |
| <b>Alternative for Key-Value Data</b> | <b>HashMap</b> when each item needs to be associated with a key.                                                                                                |
| <b>Best Practice</b>                  | Declare the variable using the interface: <code>Set&lt;String&gt; values = new HashSet&lt;&gt;();</code> rather than exposing the implementation unnecessarily. |

## Set vs HashSet

| Feature                           | Set                              | HashSet                            |
|-----------------------------------|----------------------------------|------------------------------------|
| <b>Type</b>                       | <b>Interface</b>                 | <b>Class</b>                       |
| <b>Main purpose</b>               | Defines uniqueness contract      | Implements Set using hashing       |
| <b>Can instantiate directly?</b>  | ✗ No                             | ✓ Yes                              |
| <b>Duplicate elements</b>         | ✗ Not allowed by contract        | ✗ Not allowed                      |
| <b>Ordering guarantee</b>         | Depends on implementation        | ✗ None                             |
| <b>Example</b>                    | <code>Set&lt;String&gt; s</code> | <code>new HashSet&lt;&gt;()</code> |
| <b>Implementation flexibility</b> | High                             | Specific implementation            |
| <b>Recommended declaration</b>    | <code>Set&lt;String&gt; s</code> | <code>new HashSet&lt;&gt;()</code> |

## HashSet vs Other Set Implementations

| Requirement                           | Recommended Collection        | Why                              |
|---------------------------------------|-------------------------------|----------------------------------|
| Unique elements + fast average lookup | HashSet                       | Hash-based membership operations |
| Unique elements + insertion order     | LinkedHashSet                 | Preserves insertion order        |
| Unique elements + sorted order        | TreeSet                       | Maintains sorted order           |
| Unique elements + concurrent access   | ConcurrentHashMap.newKeySet() | Concurrent set semantics         |
| Duplicates + indexed access           | ArrayList                     | Efficient index-based access     |
| Key-value association                 | HashMap                       | Designed for key-value mappings  |

## Simple Decision Table

| Requirement                                | Choose        |
|--------------------------------------------|---------------|
| "I only need unique values."               | HashSet       |
| "I need unique values in insertion order." | LinkedHashSet |
| "I need unique values sorted."             | TreeSet       |
| "I need duplicates and indexes."           | ArrayList     |
| "I need key → value mapping."              | HashMap       |

**Key point:** Set defines the **uniqueness contract**, while HashSet provides a **hash-based implementation** optimized for fast average membership, insertion, and removal operations.

| Aspect               | Map / HashMap                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
|----------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Java Version         | Java 1.2                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
| Year                 | 1998                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
| Feature              | Map / HashMap                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
| UniqueClassName      | HashMap                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
| Definition           | Map is an interface that represents a collection of <b>key-value pairs</b> . HashMap is a hash-table-based implementation that provides efficient average-time insertion, lookup, and removal by key.                                                                                                                                                                                                                                                                                   |
| Basic Syntax         | <code>Map&lt;KeyType, ValueType&gt; map = new HashMap&lt;&gt;();</code>                                                                                                                                                                                                                                                                                                                                                                                                                 |
| Underlying Structure | Hash-table-based structure using hash codes and buckets; modern implementations can use balanced trees for heavily collided buckets.                                                                                                                                                                                                                                                                                                                                                    |
| Keys                 | <b>Unique</b> ; inserting another value with an existing key replaces the previous value.                                                                                                                                                                                                                                                                                                                                                                                               |
| Values               | <b>Duplicates allowed</b>                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
| Ordering             | No guaranteed iteration order                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
| Null Key             | Allows <b>one null key</b>                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
| Null Values          | Allows multiple null values                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
| Index-Based Access   | ✗ No; access is normally through keys                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
| Typical Performance  | put(), get(), remove(), and containsKey() are typically <b>O(1) on average</b> with good hashing.                                                                                                                                                                                                                                                                                                                                                                                       |
| Thread Safety        | ✗ Not thread-safe by default                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
| Advantages           | <ol style="list-style-type: none"> <li>1. Fast average key-based lookup.</li> <li>2. Fast average insertion and removal.</li> <li>3. Stores data as meaningful key-value pairs.</li> <li>4. Keys are automatically unique.</li> <li>5. Dynamically grows as entries are added.</li> <li>6. Allows one null key and multiple null values.</li> <li>7. Generics provide compile-time type safety.</li> <li>8. Excellent general-purpose choice for lookup tables and mappings.</li> </ol> |

|                             |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
|-----------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Disadvantages</b>        | <ul style="list-style-type: none"> <li>1. Does not guarantee insertion or sorted order.</li> <li>2. Not suitable when indexed access is required.</li> <li>3. Requires appropriate equals() and hashCode() behavior for custom key classes.</li> <li>4. Uses additional memory for hash-table structures.</li> <li>5. Not thread-safe by default.</li> <li>6. Performance can degrade with poor hashing or excessive collisions.</li> <li>7. Rehashing/resizing can temporarily require additional work.</li> </ul> |
| <b>When to Use</b>          | Use HashMap when you need to <b>associate unique keys with values</b> and frequently retrieve, insert, update, or remove values using those keys.                                                                                                                                                                                                                                                                                                                                                                   |
| <b>When NOT to Use</b>      | Avoid HashMap when you require sorted keys, guaranteed insertion order, index-based access, duplicate keys, or a specialized concurrent map is required.                                                                                                                                                                                                                                                                                                                                                            |
| <b>Real-Time Use Case 1</b> | <b>Employee lookup:</b> Map an employee ID to an employee object.                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
| <b>Example 1</b>            | <code>Map&lt;Long, Employee&gt; employees = new HashMap&lt;&gt;();</code>                                                                                                                                                                                                                                                                                                                                                                                                                                           |
| <b>Real-Time Use Case 2</b> | <b>Product catalog:</b> Map a product ID to product details.                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
| <b>Example 2</b>            | <code>Map&lt;Long, Product&gt; products = new HashMap&lt;&gt;();</code>                                                                                                                                                                                                                                                                                                                                                                                                                                             |
| <b>Real-Time Use Case 3</b> | <b>User sessions:</b> Associate a session ID with session information.                                                                                                                                                                                                                                                                                                                                                                                                                                              |
| <b>Example 3</b>            | <code>Map&lt;String, UserSession&gt; sessions = new HashMap&lt;&gt;();</code>                                                                                                                                                                                                                                                                                                                                                                                                                                       |
| <b>Real-Time Use Case 4</b> | <b>Configuration properties:</b> Associate configuration names with their values.                                                                                                                                                                                                                                                                                                                                                                                                                                   |
| <b>Example 4</b>            | <code>Map&lt;String, String&gt; config = new HashMap&lt;&gt;();</code>                                                                                                                                                                                                                                                                                                                                                                                                                                              |
| <b>Real-Time Use Case 5</b> | <b>Word frequency counter:</b> Map each word to the number of times it occurs.                                                                                                                                                                                                                                                                                                                                                                                                                                      |
| <b>Example 5</b>            | <code>Map&lt;String, Integer&gt; frequency = new HashMap&lt;&gt;();</code>                                                                                                                                                                                                                                                                                                                                                                                                                                          |
| <b>Real-Time Use Case 6</b> | <b>HTTP/API response lookup:</b> Associate an API field name with its returned value.                                                                                                                                                                                                                                                                                                                                                                                                                               |
| <b>Example 6</b>            | <code>Map&lt;String, Object&gt; response = new HashMap&lt;&gt;();</code>                                                                                                                                                                                                                                                                                                                                                                                                                                            |
| <b>Real-Time Use Case 7</b> | <b>Caching:</b> Associate a resource identifier with cached data.                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
| <b>Example 7</b>            | <code>Map&lt;String, Product&gt; cache = new HashMap&lt;&gt;();</code>                                                                                                                                                                                                                                                                                                                                                                                                                                              |

|                                       |                                                                                                                                                                           |
|---------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Real-Time Use Case 8</b>           | <b>Country/currency lookup:</b> Associate a country code with a currency.                                                                                                 |
| <b>Example 8</b>                      | <code>Map&lt;String, String&gt; currencies = new HashMap&lt;&gt;();</code>                                                                                                |
| <b>Simple Example</b>                 | <code>Map&lt;Integer, String&gt; students = new HashMap&lt;&gt;();<br/>students.put(101, "Anurag");<br/>students.put(102, "Rahul");<br/>students.put(101, "Amit");</code> |
| <b>Result</b>                         | Key 101 contains "Amit" because inserting an existing key replaces its previous value "Anurag".                                                                           |
| <b>Alternate Feature</b>              | <b>LinkedHashMap</b> when insertion/access order matters; <b>TreeMap</b> when keys must remain sorted; <b>ConcurrentHashMap</b> when concurrent access is required.       |
| <b>Alternative for Duplicate Keys</b> | Use <code>Map&lt;Key, List&lt;Value&gt;&gt;</code> when one key needs to be associated with multiple values.                                                              |
| <b>Best Practice</b>                  | Program to the interface: <code>Map&lt;Long, Employee&gt; employees = new HashMap&lt;&gt;();</code>                                                                       |

## Map vs HashMap

| Feature                           | Map                                  | HashMap                            |
|-----------------------------------|--------------------------------------|------------------------------------|
| <b>Type</b>                       | <b>Interface</b>                     | <b>Class</b>                       |
| <b>Purpose</b>                    | Defines key-value mapping behavior   | Hash-based implementation of Map   |
| <b>Can instantiate directly?</b>  | ✗ No                                 | ✓ Yes                              |
| <b>Keys</b>                       | Unique                               | Unique                             |
| <b>Values</b>                     | May be duplicated                    | May be duplicated                  |
| <b>Ordering</b>                   | Depends on implementation            | No guaranteed order                |
| <b>Example</b>                    | <code>Map&lt;String, User&gt;</code> | <code>new HashMap&lt;&gt;()</code> |
| <b>Implementation flexibility</b> | High                                 | Specific implementation            |

|                                |                         |                 |
|--------------------------------|-------------------------|-----------------|
| <b>Recommended declaration</b> | Map<String, User> users | new HashMap<>() |
|--------------------------------|-------------------------|-----------------|

### HashMap vs Other Map Implementations

| Requirement                     | Recommended Collection | Why                                                                |
|---------------------------------|------------------------|--------------------------------------------------------------------|
| General key-value lookup        | HashMap                | Fast average lookup                                                |
| Preserve insertion order        | LinkedHashMap          | Maintains insertion order                                          |
| Maintain sorted keys            | TreeMap                | Keys remain sorted                                                 |
| Concurrent key-value operations | ConcurrentHashMap      | Designed for concurrent access                                     |
| Weakly referenced keys          | WeakHashMap            | Entries can be removed when keys are no longer strongly referenced |
| Enum keys                       | EnumMap                | Specialized and efficient for enum keys                            |
| Fixed/simple indexed data       | Array / ArrayList      | Better suited to positional access                                 |

### Simple Decision Table

| Requirement                                       | Choose            |
|---------------------------------------------------|-------------------|
| "I need key → value lookup."                      | HashMap           |
| "I need key → value lookup and insertion order."  | LinkedHashMap     |
| "I need key → value lookup with sorted keys."     | TreeMap           |
| "Multiple threads need to access/update the map." | ConcurrentHashMap |

|                                            |                                                                                                          |
|--------------------------------------------|----------------------------------------------------------------------------------------------------------|
| "I need duplicate keys."                   | A key cannot be duplicated in a Map; use a collection as the value if multiple values belong to one key. |
| "I need access by numeric position/index." | ArrayList / array                                                                                        |

**Key point:** Map defines the **key-value mapping contract**, while HashMap provides a **hash-based implementation** optimized for fast average key-based operations. Unlike List and Set, Map is **not a subtype of Collection**.

| Aspect               | TreeSet                                                                                                                                                                       |
|----------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Java Version         | Java 1.2                                                                                                                                                                      |
| Year                 | 1998                                                                                                                                                                          |
| Feature              | TreeSet                                                                                                                                                                       |
| UniqueClassName      | TreeSet                                                                                                                                                                       |
| Definition           | TreeSet is a sorted Set implementation that stores <b>unique elements in ascending sorted order by default</b> . It is based on a tree structure and implements NavigableSet. |
| Main Interfaces      | Set → SortedSet → NavigableSet                                                                                                                                                |
| Underlying Structure | A <b>Red-Black tree</b> (self-balancing binary search tree).                                                                                                                  |
| Duplicates           | ✗ Not allowed                                                                                                                                                                 |
| Ordering             | ✓ Sorted according to <b>natural ordering</b> or a supplied Comparator                                                                                                        |
| Insertion Order      | ✗ Not preserved; elements are ordered according to the sorting rule.                                                                                                          |
| Null Values          | Generally <b>not supported</b> in natural-order TreeSet; a NullPointerException can occur because elements must be mutually comparable.                                       |
| Index-Based Access   | ✗ Not supported                                                                                                                                                               |
| Typical Performance  | add(), remove(), and contains() are typically <b>O(log n)</b> .                                                                                                               |



|                             |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
|-----------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Thread Safety</b>        | ✗ Not synchronized by default                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
| <b>Advantages</b>           | <ol style="list-style-type: none"> <li>1. Automatically removes duplicates.</li> <li>2. Automatically maintains sorted order.</li> <li>3. Provides efficient <math>O(\log n)</math> search, insertion, and deletion.</li> <li>4. Supports navigation operations such as <code>higher()</code>, <code>lower()</code>, <code>ceiling()</code>, and <code>floor()</code>.</li> <li>5. Supports range operations such as <code>subSet()</code>, <code>headSet()</code>, and <code>tailSet()</code>.</li> <li>6. Can use custom sorting through <code>Comparator</code>.</li> <li>7. Useful when both <b>uniqueness</b> and <b>sorted order</b> are required.</li> </ol> |
| <b>Disadvantages</b>        | <ol style="list-style-type: none"> <li>1. Generally slower than <code>HashSet</code> for basic membership operations because operations are <math>O(\log n)</math> rather than average <math>O(1)</math>.</li> <li>2. Does not provide index-based access.</li> <li>3. Requires elements to be comparable or requires a suitable <code>Comparator</code>.</li> <li>4. Tree nodes introduce memory overhead.</li> <li>5. Does not preserve insertion order.</li> <li>6. Not thread-safe by default.</li> <li>7. Custom comparators must be consistent with the intended equality semantics to avoid unexpected element suppression.</li> </ol>                       |
| <b>When to Use</b>          | Use <code>TreeSet</code> when you need <b>unique elements that are automatically maintained in sorted order</b> , especially when you also need range or navigation operations.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
| <b>When NOT to Use</b>      | Avoid it when ordering is unnecessary and maximum average lookup performance is preferred ( <code>HashSet</code> ), when insertion order matters ( <code>LinkedHashSet</code> ), or when frequent index-based access is required ( <code>ArrayList</code> ).                                                                                                                                                                                                                                                                                                                                                                                                        |
| <b>Real-Time Use Case 1</b> | <b>Sorted employee salaries:</b> Maintain unique salary values in ascending order.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
| <b>Example 1</b>            | <code>TreeSet&lt;Double&gt; salaries = new TreeSet&lt;&gt;();</code>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
| <b>Real-Time Use Case 2</b> | <b>Leaderboard scores:</b> Maintain unique scores in sorted order.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
| <b>Example 2</b>            | <code>TreeSet&lt;Integer&gt; scores = new TreeSet&lt;&gt;();</code>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
| <b>Real-Time Use Case 3</b> | <b>Unique product prices:</b> Keep distinct prices automatically sorted.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
| <b>Example 3</b>            | <code>TreeSet&lt;BigDecimal&gt; prices = new TreeSet&lt;&gt;();</code>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
| <b>Real-Time Use Case 4</b> | <b>Scheduled dates:</b> Maintain unique dates in chronological order.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
| <b>Example 4</b>            | <code>TreeSet&lt;LocalDate&gt; dates = new TreeSet&lt;&gt;();</code>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |

|                                              |                                                                                                                                                                                                                             |
|----------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                              |                                                                                                                                                                                                                             |
| <b>Real-Time Use Case 5</b>                  | <b>Priority thresholds:</b> Find the next available value above or below a particular threshold.                                                                                                                            |
| <b>Example 5</b>                             | <code>scores.ceiling(80);</code>                                                                                                                                                                                            |
| <b>Real-Time Use Case 6</b>                  | <b>Price range filtering:</b> Retrieve products/prices within a particular range.                                                                                                                                           |
| <b>Example 6</b>                             | <code>prices.subSet(minPrice, maxPrice);</code>                                                                                                                                                                             |
| <b>Real-Time Use Case 7</b>                  | <b>Unique timestamps/events:</b> Maintain event timestamps in chronological order for processing or reporting.                                                                                                              |
| <b>Example 7</b>                             | <code>TreeSet&lt;Instant&gt; timestamps = new TreeSet&lt;&gt;();</code>                                                                                                                                                     |
| <b>Simple Example</b>                        | <pre>TreeSet&lt;Integer&gt; numbers = new TreeSet&lt;&gt;(); numbers.add(50); numbers.add(10); numbers.add(30); numbers.add(10);</pre>                                                                                      |
| <b>Result</b>                                | [10, 30, 50] — the duplicate 10 is ignored and the remaining values are sorted.                                                                                                                                             |
| <b>Navigation Example</b>                    | <pre>numbers.higher(30) → 50 numbers.lower(30) → 10 numbers.ceiling(30) → 30 numbers.floor(30) → 30</pre>                                                                                                                   |
| <b>Alternate Feature</b>                     | <b>HashSet</b> when sorting is unnecessary and fast average membership operations are more important.                                                                                                                       |
| <b>Alternative for Insertion Order</b>       | <b>LinkedHashSet</b> when you need unique elements while preserving insertion order.                                                                                                                                        |
| <b>Alternative for Sorted Key-Value Data</b> | <b>TreeMap</b> when you need sorted keys associated with values.                                                                                                                                                            |
| <b>Alternative for Indexed Data</b>          | <b>ArrayList</b> when duplicate elements and index-based access are required.                                                                                                                                               |
| <b>Best Practice</b>                         | Program to the interface: <code>NavigableSet&lt;Integer&gt; numbers = new TreeSet&lt;&gt;();</code> or <code>Set&lt;Integer&gt; numbers = new TreeSet&lt;&gt;();</code> depending on whether navigation methods are needed. |

## TreeSet vs Other Set Implementations

| Requirement                                    | Recommended Collection | Reason                                                                                                |
|------------------------------------------------|------------------------|-------------------------------------------------------------------------------------------------------|
| Unique elements + fast average lookup          | HashSet                | Average $O(1)$ basic operations                                                                       |
| Unique elements + insertion order              | LinkedHashSet          | Preserves insertion order                                                                             |
| Unique elements + sorted order                 | TreeSet                | Maintains sorted order                                                                                |
| Unique elements + sorted order + range queries | TreeSet                | Supports <code>subSet()</code> , <code>headSet()</code> , <code>tailSet()</code>                      |
| Unique elements + predecessor/successor lookup | TreeSet                | Supports <code>lower()</code> , <code>higher()</code> , <code>floor()</code> , <code>ceiling()</code> |
| Indexed access                                 | ArrayList              | Supports <code>get(index)</code>                                                                      |
| Key-value pairs sorted by key                  | TreeMap                | Sorted map implementation                                                                             |

## TreeSet vs HashSet vs LinkedHashSet

| Feature              | HashSet         | LinkedHashSet                | TreeSet              |
|----------------------|-----------------|------------------------------|----------------------|
| Duplicates           | ✗               | ✗                            | ✗                    |
| Insertion order      | ✗               | ✓                            | ✗                    |
| Sorted order         | ✗               | ✗                            | ✓                    |
| Average basic lookup | $O(1)$          | $O(1)$                       | $O(\log n)$          |
| Navigation methods   | ✗               | ✗                            | ✓                    |
| Range operations     | ✗               | ✗                            | ✓                    |
| Custom ordering      | ✗               | ✗                            | ✓ Comparator         |
| Best use             | Fast uniqueness | Uniqueness + insertion order | Uniqueness + sorting |

|  |  |  |  |
|--|--|--|--|
|  |  |  |  |
|--|--|--|--|

**Key point:** Choose **TreeSet** when **sorted uniqueness is the requirement**. If you only need uniqueness, HashSet is usually the simpler and faster choice; if you need uniqueness while retaining insertion order, use LinkedHashMap.

| Aspect               | TreeMap                                                                                                                                                                       |
|----------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Java Version         | Java 1.2                                                                                                                                                                      |
| Year                 | 1998                                                                                                                                                                          |
| Feature              | TreeMap                                                                                                                                                                       |
| UniqueClassName      | TreeMap                                                                                                                                                                       |
| Definition           | TreeMap is a Map implementation that stores <b>key-value pairs sorted according to the keys</b> . It implements NavigableMap and is based on a self-balancing tree structure. |
| Main Interfaces      | Map → SortedMap → NavigableMap                                                                                                                                                |
| Underlying Structure | <b>Red-Black tree</b> (self-balancing binary search tree).                                                                                                                    |
| Keys                 | Unique                                                                                                                                                                        |
| Values               | Duplicate values allowed                                                                                                                                                      |
| Key Ordering         | Natural ordering or a supplied Comparator                                                                                                                                     |
| Insertion Order      | ✗ Not preserved; entries are ordered by key                                                                                                                                   |
| Null Key             | ✗ Generally not supported when using natural ordering; a NullPointerException can occur                                                                                       |
| Null Values          | ✓ Allowed                                                                                                                                                                     |
| Index-Based Access   | ✗ Not supported                                                                                                                                                               |
| Typical Performance  | put(), get(), remove(), and key-based search operations are typically <b>O(log n)</b>                                                                                         |
| Thread Safety        | ✗ Not synchronized by default                                                                                                                                                 |
| Advantages           | 1. Automatically keeps keys sorted.                                                                                                                                           |

|                             |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
|-----------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                             | <ul style="list-style-type: none"> <li>2. Provides predictable sorted iteration order.</li> <li>3. Supports efficient <math>O(\log n)</math> insertion, lookup, and deletion.</li> <li>4. Provides navigation operations such as <code>lowerKey()</code>, <code>higherKey()</code>, <code>floorKey()</code>, and <code>ceilingKey()</code>.</li> <li>5. Supports range-based operations through <code>subMap()</code>, <code>headMap()</code>, and <code>tailMap()</code>.</li> <li>6. Supports custom key ordering with <code>Comparator</code>.</li> <li>7. Useful when both <b>key-value mapping</b> and <b>sorted navigation</b> are required.</li> </ul> |
| <b>Disadvantages</b>        | <ul style="list-style-type: none"> <li>1. Usually slower than <code>HashMap</code> for basic lookup because operations are <math>O(\log n)</math> rather than average <math>O(1)</math>.</li> <li>2. Tree nodes have additional memory overhead.</li> <li>3. Keys must be mutually comparable or a suitable <code>Comparator</code> must be provided.</li> <li>4. Does not preserve insertion order.</li> <li>5. Not thread-safe by default.</li> <li>6. More complex than <code>HashMap</code> when sorting is unnecessary.</li> </ul>                                                                                                                       |
| <b>When to Use</b>          | Use <code>TreeMap</code> when you need a <b>key-value mapping with keys maintained in sorted order</b> , particularly when you need range queries or predecessor/successor navigation.                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
| <b>When NOT to Use</b>      | Avoid it when ordering is unnecessary and fast average key lookup is the priority ( <code>HashMap</code> ), when insertion order must be preserved ( <code>LinkedHashMap</code> ), or when concurrent access requires a specialized concurrent map.                                                                                                                                                                                                                                                                                                                                                                                                           |
| <b>Real-Time Use Case 1</b> | <b>Employee directory sorted by employee ID:</b> Store employee records while keeping IDs sorted.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
| <b>Example 1</b>            | <code>Map&lt;Long, Employee&gt; employees = new TreeMap&lt;&gt;();</code>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
| <b>Real-Time Use Case 2</b> | <b>Product catalog sorted by product ID:</b> Retrieve products while iterating through IDs in sorted order.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
| <b>Example 2</b>            | <code>TreeMap&lt;Long, Product&gt; products = new TreeMap&lt;&gt;();</code>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
| <b>Real-Time Use Case 3</b> | <b>Price ranges:</b> Maintain prices as keys and retrieve entries within a specific price range.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
| <b>Example 3</b>            | <code>prices.subMap(100, 500);</code>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
| <b>Real-Time Use Case 4</b> | <b>Time-based events:</b> Map timestamps to events and process them in chronological key order.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
| <b>Example 4</b>            | <code>TreeMap&lt;Instant, Event&gt; events = new TreeMap&lt;&gt;();</code>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
| <b>Real-Time Use Case 5</b> | <b>Score/ranking thresholds:</b> Find the nearest score below or                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |

|                                          |                                                                                                                                                                                                                              |
|------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                          | above a specified value.                                                                                                                                                                                                     |
| <b>Example 5</b>                         | <code>scores.floorEntry(80);</code>                                                                                                                                                                                          |
| <b>Real-Time Use Case 6</b>              | <b>Scheduling:</b> Map scheduled times to tasks and retrieve the next scheduled task.                                                                                                                                        |
| <b>Example 6</b>                         | <code>TreeMap&lt;LocalDateTime, Task&gt; schedule = new TreeMap&lt;&gt;();</code>                                                                                                                                            |
| <b>Real-Time Use Case 7</b>              | <b>Version management:</b> Store software versions or release numbers as sorted keys.                                                                                                                                        |
| <b>Example 7</b>                         | <code>TreeMap&lt;Integer, Release&gt; releases = new TreeMap&lt;&gt;();</code>                                                                                                                                               |
| <b>Real-Time Use Case 8</b>              | <b>Financial/time-series data:</b> Maintain observations keyed by dates or timestamps for chronological navigation.                                                                                                          |
| <b>Example 8</b>                         | <code>TreeMap&lt;LocalDate, BigDecimal&gt; prices = new TreeMap&lt;&gt;();</code>                                                                                                                                            |
| <b>Simple Example</b>                    | <code>TreeMap&lt;Integer, String&gt; students = new TreeMap&lt;&gt;();</code><br><code>students.put(103, "Amit");</code><br><code>students.put(101, "Anurag");</code><br><code>students.put(102, "Rahul");</code>            |
| <b>Result</b>                            | {101=Anurag, 102=Rahul, 103=Amit} — entries are automatically ordered by key.                                                                                                                                                |
| <b>Navigation Example</b>                | <code>students.higherKey(101) → 102</code><br><code>students.lowerKey(103) → 102</code><br><code>students.ceilingKey(102) → 102</code><br><code>students.floorKey(102) → 102</code>                                          |
| <b>Alternate Feature</b>                 | <b>HashMap</b> when sorted keys are unnecessary and fast average key-based lookup is the primary requirement.                                                                                                                |
| <b>Alternative for Insertion Order</b>   | <b>LinkedHashMap</b> when entries should retain insertion order.                                                                                                                                                             |
| <b>Alternative for Concurrent Access</b> | <b>ConcurrentHashMap</b> when thread-safe concurrent access is required and sorted ordering is not needed.                                                                                                                   |
| <b>Alternative for Enum Keys</b>         | <b>EnumMap</b> when keys are enum constants.                                                                                                                                                                                 |
| <b>Best Practice</b>                     | Program to the interface: <code>NavigableMap&lt;Integer, String&gt; students = new TreeMap&lt;&gt;();</code> when navigation/range operations are required; otherwise <code>Map&lt;Integer, String&gt;</code> is sufficient. |

## Map vs TreeMap

| Feature                   | Map                                | TreeMap                           |
|---------------------------|------------------------------------|-----------------------------------|
| Type                      | Interface                          | Class                             |
| Purpose                   | Defines key-value mapping behavior | Provides sorted key-value mapping |
| Can instantiate directly? | ✗ No                               | ✓ Yes                             |
| Keys                      | Unique                             | Unique                            |
| Values                    | Can be duplicated                  | Can be duplicated                 |
| Ordering                  | Depends on implementation          | ✓ Sorted by key                   |
| Navigation                | Depends on implementation          | ✓ Supported                       |
| Range operations          | Depends on implementation          | ✓ Supported                       |
| Example                   | Map<Integer, String>               | new TreeMap<>()                   |

## TreeMap vs Other Map Implementations

| Requirement                 | Recommended Map   | Reason                                            |
|-----------------------------|-------------------|---------------------------------------------------|
| Fast average key lookup     | HashMap           | Average $O(1)$ basic operations                   |
| Preserve insertion order    | LinkedHashMap     | Maintains insertion order                         |
| Sort entries by key         | TreeMap           | Maintains sorted keys                             |
| Sorted keys + range queries | TreeMap           | subMap(), headMap(), tailMap()                    |
| Find nearest key            | TreeMap           | floorKey(), ceilingKey(), lowerKey(), higherKey() |
| Concurrent access           | ConcurrentHashMap | Designed for concurrent use                       |

|           |         |                           |
|-----------|---------|---------------------------|
| Enum keys | EnumMap | Specialized for enum keys |
|-----------|---------|---------------------------|

### Simple Decision Table

| Requirement                                                   | Choose            |
|---------------------------------------------------------------|-------------------|
| "I need key → value lookup."                                  | HashMap           |
| "I need key → value lookup + insertion order."                | LinkedHashMap     |
| "I need key → value lookup + sorted keys."                    | TreeMap           |
| "I need sorted keys + range queries."                         | TreeMap           |
| "I need the nearest key to a given key."                      | TreeMap           |
| "I need concurrent key-value access without sorted ordering." | ConcurrentHashMap |
| "My keys are enum constants."                                 | EnumMap           |

**Key point:** TreeMap is the right choice when the requirement is “**map values to unique keys and keep those keys sorted.**” If sorting is not required, HashMap is generally the better general-purpose choice.

| Aspect          | Iterator                                                                                                                                          |
|-----------------|---------------------------------------------------------------------------------------------------------------------------------------------------|
| Java Version    | Java 1.2                                                                                                                                          |
| Year            | 1998                                                                                                                                              |
| Feature         | Iterator                                                                                                                                          |
| UniqueClassName | Iterator                                                                                                                                          |
| Definition      | Iterator is an interface used to <b>traverse elements of a collection sequentially</b> without exposing the collection's internal representation. |
| Package         | java.util                                                                                                                                         |



|                             |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
|-----------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                             |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
| <b>Main Methods</b>         | hasNext(), next(), and remove()                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
| <b>Traversal Direction</b>  | Forward only                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
| <b>Works With</b>           | Most Java collections such as ArrayList, LinkedList, HashSet, TreeSet, etc.                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
| <b>Index Required?</b>      | ✗ No                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
| <b>Duplicate Handling</b>   | Depends on the underlying collection; Iterator itself does not control duplicates.                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
| <b>Ordering</b>             | Depends on the collection being traversed.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
| <b>Modification Support</b> | remove() can remove the last element returned by the iterator, when supported by the underlying collection.                                                                                                                                                                                                                                                                                                                                                                                                                                            |
| <b>Fail-Fast Behavior</b>   | Many standard collection iterators are <b>fail-fast</b> : structural modification outside the iterator during iteration may cause ConcurrentModificationException. This behavior is generally best-effort, not a synchronization mechanism.                                                                                                                                                                                                                                                                                                            |
| <b>Advantages</b>           | <ol style="list-style-type: none"> <li>1. Provides a common traversal mechanism for different collections.</li> <li>2. Does not require knowledge of the collection's internal structure.</li> <li>3. Supports sequential traversal.</li> <li>4. Can safely remove the current element using Iterator.remove() when supported.</li> <li>5. Works with collections that do not provide index-based access, such as HashSet.</li> <li>6. Helps write collection-independent code.</li> <li>7. Can be used with generic types for type safety.</li> </ol> |
| <b>Disadvantages</b>        | <ol style="list-style-type: none"> <li>1. Primarily supports forward traversal only.</li> <li>2. No direct index access.</li> <li>3. Cannot directly move backward; ListIterator is needed for lists.</li> <li>4. More verbose than the enhanced for loop for simple traversal.</li> <li>5. Structural modification outside the iterator can cause ConcurrentModificationException with fail-fast iterators.</li> <li>6. Iterator behavior depends on the underlying collection.</li> </ol>                                                            |
| <b>When to Use</b>          | Use Iterator when you need <b>explicit control over collection traversal</b> , especially when you need to remove elements safely during iteration or when working with collections without indexes.                                                                                                                                                                                                                                                                                                                                                   |





## Iterator vs ListIterator

| Feature                        | Iterator                     | ListIterator                         |
|--------------------------------|------------------------------|--------------------------------------|
| Forward traversal              | ✓                            | ✓                                    |
| Backward traversal             | ✗                            | ✓                                    |
| hasNext()                      | ✓                            | ✓                                    |
| next()                         | ✓                            | ✓                                    |
| remove()                       | ✓                            | ✓                                    |
| add()                          | ✗                            | ✓                                    |
| set()                          | ✗                            | ✓                                    |
| Index information              | ✗                            | ✓ nextIndex() / previousIndex()      |
| Works with general collections | ✓                            | ✗ Lists only                         |
| Best use                       | General collection traversal | Advanced list traversal/modification |

## Simple Decision Table

| Requirement                                   | Choose               |
|-----------------------------------------------|----------------------|
| "I just need to read every element."          | Enhanced for loop    |
| "I need to explicitly control traversal."     | Iterator             |
| "I need to remove elements while traversing." | Iterator             |
| "I need forward and backward list traversal." | ListIterator         |
| "I need filtering/mapping/aggregation."       | Stream API           |
| "I need index-based access."                  | for loop / ArrayList |

**Key point:** Iterator provides a **standard, collection-independent way to traverse elements**, and its most important practical advantage over a simple for-each loop is that it provides an explicit `remove()` operation for supported collections.

| Aspect                   | Comparable                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
|--------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Java Version             | Java 1.2                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
| Year                     | 1998                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
| Feature                  | Comparable                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
| UniqueClassName          | Comparable                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
| Definition               | Comparable<T> is an interface used to define the <b>natural ordering</b> of objects of a class. A class implements Comparable and defines its ordering through the <code>compareTo()</code> method.                                                                                                                                                                                                                                                                                                                                |
| Package                  | java.lang                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
| Main Method              | <code>int compareTo(T other)</code>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
| Return Value             | Negative value → current object is smaller; 0 → considered equal for ordering; positive value → current object is larger.                                                                                                                                                                                                                                                                                                                                                                                                          |
| Natural Ordering         | The ordering defined by the class itself.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
| Number of Natural Orders | Normally <b>one primary natural ordering</b> per class.                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
| Used By                  | <code>Collections.sort()</code> , <code>List.sort(null)</code> , <code>Arrays.sort()</code> , <code>TreeSet</code> , <code>TreeMap</code> and other APIs that rely on natural ordering.                                                                                                                                                                                                                                                                                                                                            |
| Advantages               | <ol style="list-style-type: none"><li>1. Provides a standard natural ordering for objects.</li><li>2. Makes objects directly sortable by Java's sorting APIs.</li><li>3. Works naturally with sorted collections such as <code>TreeSet</code> and <code>TreeMap</code>.</li><li>4. Keeps the default comparison logic inside the class whose objects are being compared.</li><li>5. Generic Comparable&lt;T&gt; provides compile-time type safety.</li><li>6. Makes reusable domain objects easier to sort consistently.</li></ol> |
| Disadvantages            | <ol style="list-style-type: none"><li>1. A class generally has only one natural ordering.</li><li>2. Adding or changing natural ordering can affect existing code that depends on it.</li></ol>                                                                                                                                                                                                                                                                                                                                    |

|                             |                                                                                                                                                                                                                                                                                                                                                                                                                    |
|-----------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                             | <p>3. The class itself must be modified to implement Comparable.</p> <p>4. Poorly designed compareTo() implementations can produce incorrect sorting or collection behavior.</p> <p>5. compareTo() should normally be consistent with equals(); inconsistency can cause surprising behavior in sorted collections.</p> <p>6. Not ideal when different parts of an application require different sorting rules.</p> |
| <b>When to Use</b>          | Use Comparable when a class has a <b>clear, intrinsic, and generally accepted natural ordering</b> .                                                                                                                                                                                                                                                                                                               |
| <b>When NOT to Use</b>      | Avoid Comparable when there are multiple equally valid ways to sort the same class, when you cannot modify the class, or when ordering should be defined externally.                                                                                                                                                                                                                                               |
| <b>Real-Time Use Case 1</b> | <b>Employee sorting by employee ID:</b> Define employee ID as the natural ordering.                                                                                                                                                                                                                                                                                                                                |
| <b>Example 1</b>            | <pre>class Employee implements Comparable&lt;Employee&gt; {     public int compareTo(Employee e) {         return Long.compare(id, e.id);     } }</pre>                                                                                                                                                                                                                                                            |
| <b>Real-Time Use Case 2</b> | <b>Student ranking:</b> Define students' natural ordering by marks.                                                                                                                                                                                                                                                                                                                                                |
| <b>Example 2</b>            | <pre>return Integer.compare(this.marks, other.marks);</pre>                                                                                                                                                                                                                                                                                                                                                        |
| <b>Real-Time Use Case 3</b> | <b>Product sorting by product code:</b> Products can have a natural ordering based on their code.                                                                                                                                                                                                                                                                                                                  |
| <b>Example 3</b>            | <pre>return this.code.compareTo(other.code);</pre>                                                                                                                                                                                                                                                                                                                                                                 |
| <b>Real-Time Use Case 4</b> | <b>Invoice sorting by invoice number:</b> Invoice objects can define invoice number as their natural ordering.                                                                                                                                                                                                                                                                                                     |
| <b>Example 4</b>            | <pre>return this.invoiceNo.compareTo(other.invoiceNo);</pre>                                                                                                                                                                                                                                                                                                                                                       |
| <b>Real-Time Use Case 5</b> | <b>Version objects:</b> Software versions can define their natural ordering.                                                                                                                                                                                                                                                                                                                                       |
| <b>Example 5</b>            | <pre>versionList.sort(null);</pre>                                                                                                                                                                                                                                                                                                                                                                                 |
| <b>Real-Time Use Case 6</b> | <b>Priority objects:</b> If a domain object has one obvious priority ordering, Comparable can define it.                                                                                                                                                                                                                                                                                                           |
| <b>Example 6</b>            | <pre>return Integer.compare(this.priority, other.priority);</pre>                                                                                                                                                                                                                                                                                                                                                  |

|                            |                                                                                                                                                                                                                                                  |
|----------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Simple Example</b>      | <pre> class Student implements Comparable&lt;Student&gt; {     int rollNo;     Student(int rollNo) { this.rollNo = rollNo; }     public int compareTo(Student other) {         return Integer.compare(this.rollNo, other.rollNo);     } } </pre> |
| <b>Sorting Example</b>     | <code>Collections.sort(students);</code>                                                                                                                                                                                                         |
| <b>Result</b>              | Students are sorted according to the natural ordering defined by <code>compareTo()</code> .                                                                                                                                                      |
| <b>TreeSet Example</b>     | <code>Set&lt;Student&gt; students = new TreeSet&lt;&gt;();</code> Elements are ordered according to their natural ordering.                                                                                                                      |
| <b>Alternate Feature</b>   | <b>Comparator</b> — defines an external/custom ordering without requiring the class itself to implement <code>Comparable</code> .                                                                                                                |
| <b>Best Alternative</b>    | Use <code>Comparator</code> when you need <b>multiple sorting strategies</b> , such as sorting employees by name, salary, department, or joining date.                                                                                           |
| <b>Alternative Example</b> | <code>employees.sort(Comparator.comparing(Employee::getName));</code>                                                                                                                                                                            |
| <b>Best Practice</b>       | Use <code>Comparable</code> for the class's <b>one obvious natural ordering</b> ; use <code>Comparator</code> for alternative or context-specific orderings.                                                                                     |

## Comparable vs Comparator

| Feature                         | Comparable                             | Comparator                                             |
|---------------------------------|----------------------------------------|--------------------------------------------------------|
| <b>Purpose</b>                  | Defines <b>natural ordering</b>        | Defines <b>custom/external ordering</b>                |
| <b>Main method</b>              | <code>compareTo()</code>               | <code>compare()</code>                                 |
| <b>Package</b>                  | <code>java.lang</code>                 | <code>java.util</code>                                 |
| <b>Implemented by</b>           | The class being sorted                 | Separate comparator/class, lambda, or method reference |
| <b>Modifies original class?</b> | Usually <b>yes</b>                     | <b>No</b>                                              |
| <b>Number of orderings</b>      | Typically one                          | <b>Multiple</b>                                        |
| <b>Example</b>                  | <code>class Employee implements</code> | <code>Comparator&lt;Employee&gt;</code>                |

|                                  |                                     |                                      |
|----------------------------------|-------------------------------------|--------------------------------------|
|                                  | Comparable<Employee>                |                                      |
| <b>Best for</b>                  | Intrinsic/default ordering          | Context-specific ordering            |
| <b>Used by TreeSet / TreeMap</b> | Yes, when no comparator is supplied | Yes, when a comparator is supplied   |
| <b>Example sorting</b>           | Collections.sort(list)              | list.sort(Comparator.comparing(...)) |

### When to Choose Which

| Requirement                                  | Choose            | Example                             |
|----------------------------------------------|-------------------|-------------------------------------|
| <b>One obvious natural order</b>             | <b>Comparable</b> | Employee → employee ID              |
| <b>Sort by different fields</b>              | <b>Comparator</b> | Employee → name, salary, department |
| <b>Cannot modify the original class</b>      | <b>Comparator</b> | Third-party Employee class          |
| <b>Need multiple sorting strategies</b>      | <b>Comparator</b> | Product → price, rating, name       |
| <b>Domain object has an intrinsic order</b>  | <b>Comparable</b> | Version → version number            |
| <b>Temporary/custom sorting</b>              | <b>Comparator</b> | Sort products by current discount   |
| <b>Sorted TreeSet using natural ordering</b> | <b>Comparable</b> | new TreeSet<>()                     |
| <b>Sorted TreeSet with custom ordering</b>   | <b>Comparator</b> | new TreeSet<>(comparator)           |

### Important compareTo() Rule

| Result of compareTo() | Meaning                                             |
|-----------------------|-----------------------------------------------------|
| <b>&lt; 0</b>         | Current object comes <b>before</b> the other object |



|               |                                                    |
|---------------|----------------------------------------------------|
| <b>0</b>      | Objects are considered <b>equal for ordering</b>   |
| <b>&gt; 0</b> | Current object comes <b>after</b> the other object |

### Example:

```

class Employee implements Comparable<Employee> {
    private final int id;

    Employee(int id) {
        this.id = id;
    }

    @Override
    public int compareTo(Employee other) {
        return Integer.compare(this.id, other.id);
    }
}

```

Here, Employee objects have a natural ordering based on id.

**Key point: Comparable = "How should objects of this class naturally be ordered?"**

**Comparator = "How should I order these objects for this particular situation?"**

| Aspect          | Comparator                                                                                                                                                                           |
|-----------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Java Version    | Java 1.2                                                                                                                                                                             |
| Year            | 1998                                                                                                                                                                                 |
| Feature         | Comparator                                                                                                                                                                           |
| UniqueClassName | Comparator                                                                                                                                                                           |
| Definition      | Comparator<T> is an interface used to define a <b>custom or external ordering</b> for objects. It allows objects to be sorted without requiring their class to implement Comparable. |
| Package         | java.util                                                                                                                                                                            |
| Main Method     | int compare(T o1, T o2)                                                                                                                                                              |
| Purpose         | Defines how two objects should be compared for a particular                                                                                                                          |

|                                                 |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
|-------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                                 | sorting requirement.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
| <b>Natural Ordering</b>                         | ✗ Does not define the class's natural ordering; it provides an external ordering.                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
| <b>Multiple Sorting Rules</b>                   | ✓ Multiple comparators can be created for the same class.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
| <b>Requires Modification of Original Class?</b> | ✗ No                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
| <b>Used By</b>                                  | Collections.sort(), List.sort(), Arrays.sort(), TreeSet, TreeMap, priority queues, and other APIs supporting custom ordering.                                                                                                                                                                                                                                                                                                                                                                                                                       |
| <b>Return Value</b>                             | Negative → o1 comes before o2. 0 → equal for this ordering. Positive → o1 comes after o2.                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
| <b>Advantages</b>                               | <ol style="list-style-type: none"> <li>1. Allows custom sorting without modifying the original class.</li> <li>2. Supports multiple sorting strategies for the same type.</li> <li>3. Separates comparison logic from domain/data classes.</li> <li>4. Useful for third-party classes that cannot be modified.</li> <li>5. Works with lambda expressions and method references in modern Java.</li> <li>6. Can combine multiple comparison criteria using thenComparing().</li> <li>7. Useful for context-specific sorting requirements.</li> </ol> |
| <b>Disadvantages</b>                            | <ol style="list-style-type: none"> <li>1. Too many comparators can increase code complexity.</li> <li>2. Poorly designed comparison logic can produce incorrect ordering.</li> <li>3. Comparator logic may be duplicated if not centralized or reused.</li> <li>4. More code than relying on an existing natural ordering.</li> <li>5. In sorted collections, a comparator that treats distinct objects as equal can cause one of them not to be stored.</li> </ol>                                                                                 |
| <b>When to Use</b>                              | Use Comparator when you need <b>custom, external, or multiple sorting orders</b> , or when the class cannot/should not implement Comparable.                                                                                                                                                                                                                                                                                                                                                                                                        |
| <b>When NOT to Use</b>                          | Avoid creating a separate Comparator when the class already has a clear natural ordering and that ordering is exactly what you need. In such cases, Comparable may be simpler.                                                                                                                                                                                                                                                                                                                                                                      |
| <b>Real-Time Use Case 1</b>                     | <b>Employees by salary:</b> Sort employees from lowest to highest salary.                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
| <b>Example 1</b>                                | employees.sort(Comparator.comparing(Employee::getSalary));                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
| <b>Real-Time Use Case 2</b>                     | <b>Employees by name:</b> Sort employees alphabetically without changing the Employee class's natural ordering.                                                                                                                                                                                                                                                                                                                                                                                                                                     |

|                                         |                                                                                                                                  |
|-----------------------------------------|----------------------------------------------------------------------------------------------------------------------------------|
|                                         |                                                                                                                                  |
| <b>Example 2</b>                        | <code>employees.sort(Comparator.comparing(Employee::getName));</code>                                                            |
| <b>Real-Time Use Case 3</b>             | <b>Products by price:</b> Sort products from cheapest to most expensive.                                                         |
| <b>Example 3</b>                        | <code>products.sort(Comparator.comparing(Product::getPrice));</code>                                                             |
| <b>Real-Time Use Case 4</b>             | <b>Products by rating:</b> Sort products from highest-rated to lowest-rated.                                                     |
| <b>Example 4</b>                        | <code>products.sort(Comparator.comparing(Product::getRating).reversed());</code>                                                 |
| <b>Real-Time Use Case 5</b>             | <b>Students by marks:</b> Sort students by marks, then by name when marks are equal.                                             |
| <b>Example 5</b>                        | <code>students.sort(Comparator.comparing(Student::getMarks).reversed().thenComparing(Student::getName));</code>                  |
| <b>Real-Time Use Case 6</b>             | <b>Orders by date:</b> Sort orders chronologically based on order date.                                                          |
| <b>Example 6</b>                        | <code>orders.sort(Comparator.comparing(Order::getOrderDate));</code>                                                             |
| <b>Real-Time Use Case 7</b>             | <b>Priority processing:</b> Sort tasks according to priority.                                                                    |
| <b>Example 7</b>                        | <code>tasks.sort(Comparator.comparing(Task::getPriority));</code>                                                                |
| <b>Real-Time Use Case 8</b>             | <b>Third-party objects:</b> Sort objects from a library whose source code you cannot modify.                                     |
| <b>Example 8</b>                        | Create a <code>Comparator&lt;ThirdPartyObject&gt;</code> externally and pass it to the sorting API.                              |
| <b>Simple Example</b>                   | <code>Comparator&lt;String&gt; byLength =<br/>Comparator.comparingInt(String::length);names.sort(byLength);</code>               |
| <b>Result</b>                           | Strings are sorted according to their length rather than their natural alphabetical ordering.                                    |
| <b>Multiple Criteria Example</b>        | <code>employees.sort(Comparator.comparing(Employee::getDepartment)&amp;nbsp;&amp;nbsp; thenComparing(Employee::getName));</code> |
| <b>Alternate Feature</b>                | <b>Comparable</b> — appropriate when a class has one clear, intrinsic natural ordering.                                          |
| <b>Alternative for Simple Traversal</b> | Enhanced for loop — if sorting/comparison is not actually required and you only need to traverse the collection.                 |

|                                                  |                                                                                                                                                                  |
|--------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                                  |                                                                                                                                                                  |
| <b>Alternative for Special Priority Ordering</b> | PriorityQueue — when the main requirement is repeatedly retrieving the highest/lowest-priority element rather than sorting an entire collection.                 |
| <b>Best Practice</b>                             | Use Comparator for <b>external or context-specific ordering</b> and keep reusable comparison logic in well-named comparator constants/methods where appropriate. |

## Comparator vs Comparable

| Feature                         | Comparator                  | Comparable                                              |
|---------------------------------|-----------------------------|---------------------------------------------------------|
| <b>Main purpose</b>             | Custom/external ordering    | Natural ordering                                        |
| <b>Main method</b>              | compare()                   | compareTo()                                             |
| <b>Package</b>                  | java.util                   | java.lang                                               |
| <b>Implemented by</b>           | Separate comparator         | Class being compared                                    |
| <b>Modify original class?</b>   | ✗ No                        | ✓ Usually yes                                           |
| <b>Multiple orderings</b>       | ✓ Yes                       | Typically ✗ One natural ordering                        |
| <b>Third-party class</b>        | ✓ Excellent choice          | ✗ Cannot implement without modifying/wrapping the class |
| <b>Context-specific sorting</b> | ✓ Excellent                 | ✗ Not ideal                                             |
| <b>Lambda support</b>           | ✓ Yes                       | Not applicable in the same way                          |
| <b>Example</b>                  | Comparator<Employee>        | Employee implements Comparable<Employee>                |
| <b>Best use</b>                 | Employee → salary/name/date | Employee → employee ID as natural order                 |

## Common Comparator Operations

| Operation | Example | Purpose |
|-----------|---------|---------|
|-----------|---------|---------|

|                              |                                                        |                                  |
|------------------------------|--------------------------------------------------------|----------------------------------|
| <b>Compare integers</b>      | <code>Comparator.comparingInt(Employee::getAge)</code> | Sort by integer property         |
| <b>Compare strings</b>       | <code>Comparator.comparing(Employee::getName)</code>   | Alphabetical sorting             |
| <b>Reverse order</b>         | <code>.reversed()</code>                               | Descending order                 |
| <b>Multiple criteria</b>     | <code>.thenComparing(...)</code>                       | Secondary/tertiary sorting       |
| <b>Null values first</b>     | <code>Comparator.nullsFirst(...)</code>                | Handle null values               |
| <b>Null values last</b>      | <code>Comparator.nullsLast(...)</code>                 | Handle null values               |
| <b>Natural order</b>         | <code>Comparator.naturalOrder()</code>                 | Use an object's natural ordering |
| <b>Reverse natural order</b> | <code>Comparator.reverseOrder()</code>                 | Reverse natural ordering         |

#### Real-World Decision Table

| Requirement                                        | Choose     |
|----------------------------------------------------|------------|
| Class has one obvious natural ordering             | Comparable |
| Need multiple ways to sort the same class          | Comparator |
| Need to sort by salary today and name tomorrow     | Comparator |
| Cannot modify the class                            | Comparator |
| Third-party library class                          | Comparator |
| Sorting depends on user-selected criteria          | Comparator |
| Domain object has one universally meaningful order | Comparable |
| Need multi-level sorting                           | Comparator |
| Need sorted TreeSet/TreeMap using custom ordering  | Comparator |

**Key point:** Comparable answers “What is this object's natural order?”, while **Comparator** answers “How do I want to order these objects in this particular situation?”. For modern Java applications, Comparator is especially powerful because comparison rules can be composed with methods such as `comparing()`, `thenComparing()`, and `reversed()`.

| Aspect             | Swing UI                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
|--------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Java Version       | Java 1.2                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
| Year               | 1998                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
| Feature            | Swing UI                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
| UniqueClassName    | SwingIntro                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
| Definition         | Swing is Java's GUI toolkit built on top of the AWT infrastructure. It provides a rich set of <b>lightweight, platform-independent GUI components</b> such as JFrame, JPanel, JButton, JLabel, JTextField, JTable, and JTree.                                                                                                                                                                                                                                                                     |
| Main Package       | javax.swing                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
| UI Model           | Event-driven desktop GUI programming                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
| Component Examples | JFrame, JDialog, JPanel, JButton, JLabel, JTextField, JTextArea, JTable, JTree, JComboBox                                                                                                                                                                                                                                                                                                                                                                                                         |
| Look and Feel      | Supports pluggable <b>Look &amp; Feel</b> , allowing the appearance of Swing applications to be changed without fundamentally changing application logic.                                                                                                                                                                                                                                                                                                                                         |
| Platform Support   | Cross-platform because Swing components are primarily implemented in Java rather than relying directly on native widgets.                                                                                                                                                                                                                                                                                                                                                                         |
| Advantages         | <ol style="list-style-type: none"> <li>1. Cross-platform desktop GUI support.</li> <li>2. Large collection of ready-made UI components.</li> <li>3. Pluggable Look &amp; Feel.</li> <li>4. Highly customizable components.</li> <li>5. Supports advanced components such as JTable, JTree, and JTabbedPane.</li> <li>6. Mature and stable technology.</li> <li>7. Integrates well with Java's event-handling model.</li> <li>8. Can be used without requiring a browser or web server.</li> </ol> |

|                             |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
|-----------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Disadvantages</b>        | <ol style="list-style-type: none"> <li>1. Primarily intended for <b>desktop applications</b>, not modern web/mobile UIs.</li> <li>2. UI development can be verbose compared with modern declarative UI frameworks.</li> <li>3. Requires careful handling of the <b>Event Dispatch Thread (EDT)</b>.</li> <li>4. Swing's default visual style can appear dated compared with modern UI toolkits.</li> <li>5. Complex applications can require significant UI architecture and state-management code.</li> <li>6. Hardware-accelerated/modern graphics requirements may favor newer frameworks in some scenarios.</li> <li>7. Not the preferred choice for new cross-platform mobile or browser-based applications.</li> </ol> |
| <b>When to Use</b>          | Use Swing when building or maintaining <b>Java desktop applications</b> , especially existing enterprise/engineering tools, internal utilities, administration applications, and applications that benefit from mature desktop widgets.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
| <b>When NOT to Use</b>      | Avoid Swing for new browser-based applications, Android/iOS applications, or projects requiring modern web/mobile UI technologies. For a new Java desktop application, also evaluate <b>JavaFX</b> before choosing Swing.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
| <b>Real-Time Use Case 1</b> | <b>Desktop inventory system:</b> Create forms for adding products and tables for viewing inventory.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
| <b>Example 1</b>            | <code>JFrame frame = new JFrame("Inventory");JTable table = new JTable(data, columns);</code>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
| <b>Real-Time Use Case 2</b> | <b>Banking/financial desktop application:</b> Provide account-management forms and transaction tables.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
| <b>Example 2</b>            | <code>JTextField accountNo = new JTextField();JButton transfer = new JButton("Transfer");</code>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
| <b>Real-Time Use Case 3</b> | <b>Employee management system:</b> Create employee entry forms and searchable employee tables.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
| <b>Example 3</b>            | <code>JButton save = new JButton("Save");JTable employees = new JTable();</code>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
| <b>Real-Time Use Case 4</b> | <b>Database administration tool:</b> Provide desktop screens for browsing tables, executing queries, and displaying results.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
| <b>Example 4</b>            | <code>JTextArea sql = new JTextArea();JTable results = new JTable();</code>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
| <b>Real-Time Use Case 5</b> | <b>POS/retail desktop application:</b> Build product selection, billing, and receipt interfaces.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |

|                                                    |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
|----------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                                    |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
| <b>Example 5</b>                                   | <code>JComboBox&lt;String&gt; products = new JComboBox&lt;&gt;();</code><br><code>checkout = new JButton("Checkout");</code>                                                                                                                                                                                                                                                                                                                                                                         |
| <b>Real-Time Use Case 6</b>                        | <b>Engineering/scientific tool:</b> Create desktop interfaces for configuring calculations and displaying results.                                                                                                                                                                                                                                                                                                                                                                                   |
| <b>Example 6</b>                                   | <code>JPanel settings = new JPanel();</code><br><code>JButton calculate = new JButton("Calculate");</code>                                                                                                                                                                                                                                                                                                                                                                                           |
| <b>Real-Time Use Case 7</b>                        | <b>Internal administration utility:</b> Build forms for managing configuration, users, or system settings.                                                                                                                                                                                                                                                                                                                                                                                           |
| <b>Example 7</b>                                   | <code>JFrame frame = new JFrame("Administration");</code>                                                                                                                                                                                                                                                                                                                                                                                                                                            |
| <b>Simple Example</b>                              | <code>SwingUtilities.invokeLater(() -&gt; {</code><br><code>    JFrame frame = new JFrame("My Application");</code><br><code>    JButton button = new JButton("Click Me");</code><br><code>    button.addActionListener(e -&gt; System.out.println("Clicked"));</code><br><code>    frame.add(button);</code><br><code>    frame.setSize(300, 200);</code><br><code>    frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);</code><br><code>    frame.setVisible(true);</code><br><code>});</code> |
| <b>Important Threading Rule</b>                    | Swing components should generally be created and interacted with on the <b>Event Dispatch Thread (EDT)</b> .<br><code>SwingUtilities.invokeLater()</code> is commonly used to schedule UI work on the EDT.                                                                                                                                                                                                                                                                                           |
| <b>Alternate Feature</b>                           | <b>JavaFX</b> is the main modern Java desktop UI alternative for applications where a newer UI toolkit is desired.                                                                                                                                                                                                                                                                                                                                                                                   |
| <b>Other Alternatives</b>                          | <b>AWT</b> for simpler/older native GUI components; web UI frameworks for browser applications; platform-specific UI frameworks for native mobile applications.                                                                                                                                                                                                                                                                                                                                      |
| <b>Best Alternative for Modern Java Desktop UI</b> | <b>JavaFX</b> , especially when you need modern styling, richer graphics, CSS-based UI customization, FXML, animation, and multimedia capabilities.                                                                                                                                                                                                                                                                                                                                                  |
| <b>Best Practice</b>                               | Keep UI updates on the EDT, separate UI from business logic, use appropriate layout managers instead of fixed positioning, and avoid blocking long-running operations on the EDT.                                                                                                                                                                                                                                                                                                                    |

## Swing vs Common Alternatives



| Requirement                        | Recommended Technology         | Reason                                         |
|------------------------------------|--------------------------------|------------------------------------------------|
| Existing Swing desktop application | Swing                          | Mature ecosystem and compatibility             |
| New Java desktop application       | JavaFX                         | More modern desktop UI capabilities            |
| Very simple/legacy Java GUI        | AWT                            | Direct, simpler native GUI foundation          |
| Browser-based application          | Web technologies               | Runs in the browser                            |
| Android application                | Android UI toolkit             | Designed for Android                           |
| iOS application                    | Apple platform UI technologies | Designed for iOS                               |
| Rich Java desktop graphics/media   | JavaFX                         | Better fit for modern rich-client requirements |

### Swing vs AWT

| Feature                | Swing                      | AWT                                  |
|------------------------|----------------------------|--------------------------------------|
| Introduced             | Java 1.2-era platform      | Java 1.0                             |
| Package                | javax.swing                | java.awt                             |
| Component type         | Mostly lightweight         | Mostly heavyweight/native-peer based |
| Components             | Richer set                 | More basic                           |
| Look & Feel            | Pluggable                  | More platform-dependent              |
| Customization          | High                       | More limited                         |
| JTable / JTree         | ✓                          | ✗                                    |
| Desktop GUI            | ✓                          | ✓                                    |
| Modern new development | Usually legacy/maintenance | Mostly legacy/simple cases           |

|  |  |  |
|--|--|--|
|  |  |  |
|--|--|--|

## Swing vs JavaFX

| Feature                      | Swing                                      | JavaFX                                |
|------------------------------|--------------------------------------------|---------------------------------------|
| Primary purpose              | Java desktop GUI                           | Modern Java desktop GUI               |
| UI components                | Extensive                                  | Extensive                             |
| Styling                      | Look & Feel, UI delegates                  | <b>CSS support</b>                    |
| Declarative UI               | Limited                                    | <b>FXML available</b>                 |
| Modern graphics              | Good but older architecture                | Stronger modern graphics capabilities |
| Animation                    | Supported                                  | Strong support                        |
| Multimedia                   | Limited/legacy-oriented                    | Richer support                        |
| Existing legacy applications | <b>Excellent choice</b>                    | Migration may require redesign        |
| New desktop projects         | Consider when maintaining existing systems | <b>Often preferable to Swing</b>      |

**Key point: Swing is a mature Java desktop GUI toolkit introduced in the Java 1.2 era.** It remains important for maintaining existing Java desktop applications, but for a **new Java desktop project**, JavaFX is generally the more modern alternative.

| Aspect          | Security Permissions / Java Security Model                             |
|-----------------|------------------------------------------------------------------------|
| Java Version    | Java 1.2                                                               |
| Year            | 1998                                                                   |
| Feature         | Security permissions                                                   |
| UniqueClassName | SecurityModel                                                          |
| Definition      | Java 1.2 introduced a more flexible, <b>policy-based, fine-grained</b> |

|                         |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
|-------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                         | <b>security architecture</b> in which code could be granted specific permissions to access protected resources such as files and network connections. ( <a href="#">NDSS Symposium</a> )                                                                                                                                                                                                                                                                                                                                                                   |
| <b>Main Purpose</b>     | Control what code is allowed to do instead of automatically granting broad access to system resources.                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
| <b>Core Components</b>  | Permission, Policy, ProtectionDomain, AccessController, SecureClassLoader, and SecurityManager. ( <a href="#">NDSS Symposium</a> )                                                                                                                                                                                                                                                                                                                                                                                                                         |
| <b>Permission</b>       | Represents an access right to a resource, such as reading a file or connecting to a network host. A permission object represents the right; policy determines whether it is granted. ( <a href="#">Oracle Docs</a> )                                                                                                                                                                                                                                                                                                                                       |
| <b>Policy</b>           | Defines which permissions are granted to code based on characteristics such as code source and signer. ( <a href="#">Oracle Docs</a> )                                                                                                                                                                                                                                                                                                                                                                                                                     |
| <b>ProtectionDomain</b> | Associates executing classes/code with a set of permissions. ( <a href="#">Oracle Docs</a> )                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
| <b>AccessController</b> | Performs access-control decisions by examining the execution context and permissions associated with relevant protection domains. ( <a href="#">Oracle Docs</a> )                                                                                                                                                                                                                                                                                                                                                                                          |
| <b>SecurityManager</b>  | Historically provided security checks that mediated access to protected resources when installed.                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
| <b>Granularity</b>      | <b>Fine-grained</b> — permissions can be granted for particular resources and actions rather than simply granting all-or-nothing access. ( <a href="#">Novell</a> )                                                                                                                                                                                                                                                                                                                                                                                        |
| <b>Advantages</b>       | <ol style="list-style-type: none"> <li>1. Fine-grained access control.</li> <li>2. Supports the principle of least privilege.</li> <li>3. Permissions can be configured through security policies.</li> <li>4. Separates security policy from application code.</li> <li>5. Can restrict file, network, system-property and other sensitive operations.</li> <li>6. Protection domains isolate code according to its source and granted permissions.</li> <li>7. Useful for environments executing code from different or less-trusted sources.</li> </ol> |
| <b>Disadvantages</b>    | <ol style="list-style-type: none"> <li>1. Security policy configuration can be complicated.</li> <li>2. Incorrect permissions can either break legitimate functionality or grant excessive privileges.</li> <li>3. Security checks introduce some runtime overhead.</li> <li>4. Debugging permission failures can be difficult.</li> <li>5. Developers must understand policy, protection domains, permissions, and privileged operations.</li> </ol>                                                                                                      |

|                             |                                                                                                                                                                                                                                                                                                       |
|-----------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                             | 6. The historical SecurityManager-based model is now obsolete for modern Java applications: SecurityManager and related APIs were deprecated for removal by JEP 411, and later Java documentation states there is no replacement for the Security Manager. ( <a href="#">Oracle Docs</a> )            |
| <b>When to Use</b>          | Historically, use permission-based security when an application needs to <b>restrict access to sensitive resources according to code origin or security policy</b> , particularly in controlled legacy environments that still depend on the model.                                                   |
| <b>When NOT to Use</b>      | Do not design a new modern application around the old SecurityManager model. For current applications, use OS/container isolation, application-level authorization, strong authentication, filesystem permissions, network controls, sandboxing, and other modern security mechanisms as appropriate. |
| <b>Real-Time Use Case 1</b> | <b>Restricted file access:</b> Allow trusted code to read a specific directory while preventing access to other files.                                                                                                                                                                                |
| <b>Example 1</b>            | A policy can grant FilePermission with read access to a particular directory rather than granting unrestricted filesystem access.                                                                                                                                                                     |
| <b>Real-Time Use Case 2</b> | <b>Restricted network access:</b> Allow code to connect only to an approved host and port.                                                                                                                                                                                                            |
| <b>Example 2</b>            | A policy can grant SocketPermission for a specified host/port and permitted action.                                                                                                                                                                                                                   |
| <b>Real-Time Use Case 3</b> | <b>Sandboxed/untrusted code:</b> Limit code from an external source so that it cannot freely access local resources.                                                                                                                                                                                  |
| <b>Example 3</b>            | Code loaded from an untrusted source can be placed in a protection domain with limited permissions.                                                                                                                                                                                                   |
| <b>Real-Time Use Case 4</b> | <b>Plugin systems:</b> Historically, third-party plugins could be given restricted permissions rather than unrestricted application privileges.                                                                                                                                                       |
| <b>Example 4</b>            | A plugin might receive permission to read a designated application directory but not arbitrary system files.                                                                                                                                                                                          |
| <b>Real-Time Use Case 5</b> | <b>Privileged library operation:</b> Trusted system/library code can perform a narrowly defined operation on behalf of less-privileged calling code.                                                                                                                                                  |
| <b>Example 5</b>            | AccessController.doPrivileged(...) was designed for trusted code to perform an operation requiring a permission it possesses. ( <a href="#">Oracle</a>                                                                                                                                                |

|                                  |                                                                                                                                                                                                                                   |
|----------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                  | <a href="#">Docs</a> )                                                                                                                                                                                                            |
| <b>Real-Time Use Case 6</b>      | <b>Enterprise legacy applications:</b> Older Java systems can use policy-based permissions to control sensitive operations where the legacy security architecture is still deployed.                                              |
| <b>Example 6</b>                 | A legacy service can grant a component permission to access only a particular configuration file.                                                                                                                                 |
| <b>Simple Conceptual Example</b> | <code>FilePermission("/app/data/*", "read")</code> represents permission to read files under a specified location.                                                                                                                |
| <b>Important Note</b>            | Creating a <code>Permission</code> object by itself <b>does not grant access</b> ; the applicable security policy must grant the permission to the relevant code. ( <a href="#">Oracle Docs</a> )                                 |
| <b>Alternate Feature</b>         | <b>Modern application security mechanisms</b> — OS-level permissions, containers, application authorization, authentication, network policies, and sandboxing are generally more appropriate for new applications.                |
| <b>Historical Alternative</b>    | <b>Code-signing and controlled class loading</b> could complement or reduce reliance on unrestricted permissions, but they are not substitutes for complete authorization.                                                        |
| <b>Best Practice Today</b>       | Treat the old Java permission/ <code>SecurityManager</code> architecture primarily as a <b>legacy Java security feature</b> . For new systems, design security at the application, OS, container, network, and deployment layers. |

## Java 1.2 Security Permission Flow

| Step | Component                                      | Responsibility                                                             |
|------|------------------------------------------------|----------------------------------------------------------------------------|
| 1    | Code source                                    | Code is loaded from a particular location and may have signer information. |
| 2    | <code>SecureClassLoader</code> / class loading | Code is associated with a protection domain.                               |
| 3    | <code>ProtectionDomain</code>                  | Associates the code with applicable permissions.                           |
| 4    | Policy                                         | Determines what permissions the code should receive.                       |
| 5    | Application requests resource                  | Example: file read or network connection.                                  |

|   |                  |                                                                                       |
|---|------------------|---------------------------------------------------------------------------------------|
|   |                  |                                                                                       |
| 6 | Security check   | Access-control machinery evaluates the requested operation.                           |
| 7 | AccessController | Determines whether the required permission is available in the execution context.     |
| 8 | Result           | Access is <b>allowed</b> if authorized; otherwise a security exception can be raised. |

### Examples of Java Permission Types

| Permission                | Controls                            | Example                                     |
|---------------------------|-------------------------------------|---------------------------------------------|
| <b>FilePermission</b>     | File/directory access               | Read a configuration file                   |
| <b>SocketPermission</b>   | Network connections/listening       | Connect to a server                         |
| <b>PropertyPermission</b> | System properties                   | Read a system property                      |
| <b>RuntimePermission</b>  | Various runtime operations          | Certain runtime-level capabilities          |
| <b>SecurityPermission</b> | Security-related operations         | Sensitive security configuration operations |
| <b>AllPermission</b>      | Essentially unrestricted permission | Highly trusted code                         |

### Security Permissions vs Modern Alternatives

| Requirement                         | Historical Java Security Model | Modern Approach                                            |
|-------------------------------------|--------------------------------|------------------------------------------------------------|
| <b>Restrict filesystem access</b>   | FilePermission                 | OS/container filesystem permissions                        |
| <b>Restrict network access</b>      | SocketPermission               | Firewall, network policy, container/network isolation      |
| <b>Restrict third-party plugins</b> | SecurityManager/policy         | Process isolation, containers, dedicated plugin sandboxing |

|                              |                                  |                                                      |
|------------------------------|----------------------------------|------------------------------------------------------|
| <b>User authorization</b>    | Security policy                  | Application<br>RBAC/ABAC/authorization               |
| <b>Authentication</b>        | Not primarily its responsibility | OAuth/OIDC, certificates, application authentication |
| <b>Secrets protection</b>    | Permission restrictions alone    | Secret managers, environment/deployment controls     |
| <b>Application isolation</b> | Protection domains               | Containers, VMs, separate processes, OS isolation    |
| <b>Legacy Java sandbox</b>   | SecurityManager + permissions    | Migrate to modern isolation/security controls        |

### Security Permissions vs SecurityManager

| Feature                   | Security Permissions                                                        | SecurityManager                                       |
|---------------------------|-----------------------------------------------------------------------------|-------------------------------------------------------|
| <b>Role</b>               | Define/represent access rights                                              | Historically enforced/checks access                   |
| <b>Example</b>            | FilePermission                                                              | SecurityManager.checkRead()                           |
| <b>Main concept</b>       | <b>What is allowed</b>                                                      | <b>Check whether it is allowed</b>                    |
| <b>Policy involvement</b> | Yes                                                                         | Yes                                                   |
| <b>Java 1.2 relevance</b> | Core security architecture                                                  | Expanded to work with the Java 1.2 policy-based model |
| <b>Modern status</b>      | Permission APIs remain in Java, but the old SecurityManager model is legacy | <b>Deprecated for removal</b>                         |

**Key point:** Java 1.2's major security advance was moving toward **fine-grained, policy-based permissions**: *code* → *protection domain* → *permissions* → *access decision*. This was significantly more flexible than the earlier sandbox model. ([NDSS Symposium](#))

**Modern caveat:** Although the architecture is historically important for understanding Java 1.2, the SecurityManager-based sandbox is **not the recommended security architecture for new Java applications today**. ([Oracle Docs](#))